

Corner

Relatório de modificações e plano de melhorias

O que foi construído, o que deu errado e foi corrigido, como está o código hoje e o que fazer em seguida, no processo e na estrutura.

Sessão de trabalho de 05 e 06 de outubro de 2026

Base: commit d396703 + alterações ainda não enviadas

corner2.vercel.app · github.com/renan-ml22/Corner2

8	47	48
telas no app	arquivos de código · ~5.400 linhas	verificações de navegador passando

Documento gerado em 06 de outubro de 2026

VISÃO GERAL

Resumo executivo

O Corner começou a sessão como um conjunto de telas com dados fixos. Terminou como um app com acesso protegido por login, dados reais gravados no aparelho, planos de treino, cronômetro de rounds, biblioteca técnica, acompanhamento de carga e saúde e uso offline.

8
telas: Hoje, Plano, Progresso, Perfil, Treino, Técnicas, Saúde e Login

4
planos de treino prontos, de iniciante a pré-competição

48
verificações de navegador passando (scripts descartáveis)

0
testes automatizados guardados no repositório

ONDE ESTÁ	O QUE CONTÉM	SITUAÇÃO
GitHub e Vercel commit d396703	Login, planos, registro de treino salvo, telas Hoje, Plano, Progresso e Perfil com dados reais.	Enviado
Só no seu computador 44 arquivos pendentes	Cronômetro, biblioteca técnica, saúde e carga, alertas, perfil de atleta, reagendar sessão, backup, lembrete, modo offline, ícones, fluxograma e este relatório.	Não enviado
Ainda não existe	Conta com banco de dados e sincronização, notificações push, testes automatizados no repositório.	Planejado

O que está sólido

- Typecheck, lint e build passam
- Regras de cálculo isoladas em módulos puros
- Dados do usuário validados antes de usar
- Login seguro, verificado em produção

O que merece atenção

- Nenhum teste automático guardado no projeto
- 3 vulnerabilidades nas dependências (1 crítica)
- Dados só ficam no aparelho do usuário
- Segunda fase ainda não publicada nem testada em celular

O QUE ACONTECEU, EM ORDEM

Linha do tempo da sessão

05/10 Base visual e navegação

Aplicado o script de UX enviado por você. Ele estava incompleto (faltava o componente de painéis), então criei o que faltava. Barra inferior com botão "+", tokens de cor do tema escuro e remoção da barra de status falsa de iPhone.

05/10 Login moderno e seguro

Tela com logo animada, aurora de fundo e cartão de vidro. Sessão em cookie protegido, proteção de rotas, usuário de teste e senha guardada só como hash.

05–06/10 Acesso na Vercel

O login funcionava no computador e falhava no site publicado. Três causas diferentes, corrigidas em sequência (ver página 5).

06/10 Funcionalidades reais

Persistência no aparelho, 4 planos de treino, telas dinâmicas e registro salvo. Enviado ao GitHub no commit d396703.

06/10 Revisão como especialista em boxe

Apontadas 5 lacunas: cronômetro de rounds, conteúdo técnico, carga e recuperação, saúde específica do boxe e base técnica do app.

06/10 Implementação das lacunas

Cronômetro, biblioteca técnica, check-in e alertas, saúde, perfil de atleta, reagendamento, backup, lembrete e modo offline. Testado de ponta a ponta em navegador.

06/10 Práticas de trabalho definidas

TDD, subagentes em tarefas grandes e auditoria antes de entregas. Levantamento do estado real do projeto.

06/10 Documentação

Fluxograma do app em PDF (8 páginas) e este relatório, ambos na pasta `docs/`.

PARTE 1

Fundação e acesso

A base visual e o login foram feitos primeiro, porque tudo o que vem depois depende de saber quem é o usuário e de uma identidade visual consistente.

ITEM	O QUE FOI FEITO	ARQUIVOS PRINCIPAIS
Identidade visual	Tokens de cor do tema escuro, vermelho e azul dos dois corners, ícones Phosphor, área segura em celulares com entalhe.	`globals.css`, `layout.tsx`
Navegação	Barra inferior com 4 abas e botão "+" central, que abre o registro de treino em painel deslizante.	`BottomNav.tsx`, `AppSheets.tsx`
Login	Logo animada (cordas se desenham, luva entra), fundo com aurora, cartão de vidro com borda de luz, rótulos flutuantes, mostrar senha, aviso de Caps Lock, erro com tremida e botão com estado de carregamento. Respeita "reduzir movimento".	`(auth)/login/*`, `CornerLogo.tsx`
Sessão	Token assinado (JWT) em cookie `httpOnly`, 7 dias. Marcado como seguro só quando a conexão é HTTPS.	`lib/session.ts`
Proteção de rotas	Verificação rápida em `proxy.ts` e verificação definitiva no servidor, perto dos dados (camada de acesso).	`proxy.ts`, `lib/dal.ts`
Usuário de teste	Definido por variáveis de ambiente. A senha existe só como hash com sal; a comparação leva o mesmo tempo para e-mail inexistente e senha errada.	`lib/users.ts`, `.env.local`

Decisões de segurança

- Segredos fora do git (`.env.local` ignorado)
- Senha nunca em texto, nem no código
- Dois níveis de checagem de sessão
- Sair da conta apaga também as cópias offline

Decisões de design

- Cores, ícones e logo do artefato original do Corner
- Erros por formas e texto, não só por cor
- Campos com 16 px para evitar zoom no iPhone
- Animações desligam com "reduzir movimento"

O QUE DEU ERRADO

Problemas encontrados e corrigidos

PROBLEMA	CAUSA	CORREÇÃO
Script do usuário não compilava	Importava um componente que não existia e citava um segundo script não enviado.	Componente criado; layout dividido em grupos de rotas.
“E-mail ou senha incorretos” só na Vercel	O `.env.local` não vai ao GitHub, então o site publicado não tinha nenhum usuário.	4 variáveis cadastradas na Vercel e novo deploy. Log de aviso quando faltarem.
Login aceito mas voltava ao login	Cookie marcado como seguro em acesso por HTTP (IP da rede): o navegador o descartava.	Seguro apenas quando a conexão é HTTPS. Testado por IP, localhost e modo dev.
“This page couldn’t load” após deploy	Aba antiga misturava a página nova com a versão anterior do site.	Fechar as abas e recarregar sem cache. Login real em produção verificado.
Pacote do Next alterado	Um `npm install` rodou dentro da pasta de documentação do Next.	Pacote reinstalado limpo; dependências no `package.json` certo.
Build quebrado após mover rotas	Cache de tipos do modo dev apontava para pastas antigas.	Cache apagado; build e tipos passam.
Alertas nunca apareciam	A tela de notificações não tinha botão que a abrisse, e o “ajuste” era texto fixo.	Substituída por alertas calculados, com sino na tela Hoje.
Ícone do app dava erro 404	Manifesto apontava para arquivos de ícone inexistentes.	Ícones gerados a partir da logo (192, 512, versão recortável e iPhone).
Botão “+” enviava dado errado	O evento do clique era passado no lugar das opções do treino (pego pelo typecheck).	Chamada corrigida.

Nos testes de navegador também apareceu um problema só do ambiente de teste: o Chrome sem tela não grava cache em pastas com nome curto do Windows (`RENAN~1`). O app não tinha defeito; o teste passou a usar uma pasta comum.

PARTE 2

Funcionalidades reais

Do protótipo com dados fixos para um app que lembra o que você fez. Tudo ficou no aparelho, sem banco de dados externo, como combinado.

ITEM	O QUE FOI FEITO
Persistência	Hook `useStorage`: guarda no navegador, valida o que lê (dado corrompido não derruba o app), sincroniza entre abas e evita erro de hidratação. Chaves separadas por usuário.
4 planos de treino	Fundamentos (iniciante, 4 sem.), Base Aeróbica (6 sem.), Técnica Avançada (4 sem.) e Pré-competição (6 sem.). Cada sessão tem dia, tipo, rounds, descanso, descrição e checklist; o conteúdo muda por fase, e a última semana do plano de competição é de polimento.
Tela Hoje	Sessão do dia com checklist próprio, próxima sessão, sequência calculada do histórico e sugestão quando não há plano. Dia de descanso quando o plano não tem sessão.
Tela Plano	Plano ativo com progresso, próximo treino em destaque, seletor de semana e status de cada sessão (feita, hoje, a fazer, não feita), cruzando com o histórico.
Tela Progresso	Resumo do mês, gráfico de rounds das últimas 8 semanas com leitura ao passar o mouse e tabela alternativa, treinos por tipo e histórico completo.
Registro de treino	Data, tipo, rounds, minutos, esforço percebido (RPE) e duração. Um treino feito em outro dia da semana vale para a sessão do mesmo tipo.

Sequência

Calculada em **semanas seguidas na meta**, não em dias: em um plano de 3 treinos por semana, uma sequência diária quebraria a cada dia de descanso.

Duração do treino

Sem cronômetro, a duração salva é o tempo gasto preenchendo o registro, não o treino em si. Por isso o app usa rounds × minutos para o tempo de trabalho.

PARTE 3

Melhorias de especialista em boxe

Resposta às cinco lacunas apontadas na revisão. A conta com banco de dados ficou de fora, por depender de provedor e credenciais.

Cronômetro de rounds

- Preparo de 10 s, rounds e descansos
- Sino, matraca aos 10 s finais, bips 3-2-1 e vibração
- Pausar, pular fase e encerrar com confirmação
- Tela acesa; tempo pelo relógio do aparelho
- Foco de cada round por tipo e nível
- Registro com a duração real

Biblioteca técnica

- 6 golpes numerados e 6 defesas
- 18 combinações em 3 níveis
- Guias de aquecimento, bandagem e desaquecimento
- Postura ortodoxa ou canhota troca a mão da frente

Carga e recuperação

- Check-in diário: sono, energia e dor
- Ajuste sugerido do treino do dia, que você aceita ou não
- Carga = esforço × minutos; últimos 7 dias contra a média anterior
- Alertas: cabeça, lesões, sparrings seguidos, carga e meta

Saúde do boxeador

- Peso com gráfico e categoria; alerta de perda rápida
- Lesões e dores por região
- Sparring: intensidade, pancadas na cabeça, parceiro e notas
- Aviso na hora após pancadas fortes

Personalização

- Perfil de atleta em 5 passos, com plano recomendado
- Mudar o dia de uma sessão
- Meta semanal, postura e lembrete diário

Base técnica

- App instalável, com ícones da logo
- Telas visitadas e cronômetro abrem offline
- Backup em arquivo (exportar e importar, validado)
- Sair da conta apaga as cópias offline

Limite conhecido do lembrete

O lembrete só dispara com o app aberto. Avisar com o app fechado exige um servidor de notificações, que depende da conta com banco de dados.

COMO O CÓDIGO ESTÁ ORGANIZADO

Estrutura do projeto

```
corner2/
├─ app/                          19 arquivos · 2.777 linhas
│  ├─ (auth)/login/              tela de login
│  ├─ (app)/                     telas protegidas
│  │  ├─ hoje · plano · progresso · perfil
│  │  ├─ treino/                 cronômetro de rounds
│  │  ├─ tecnicas/               biblioteca técnica
│  │  └─ saude/                  peso, lesões, cabeça
│  ├─ actions/auth.ts            login e logout (servidor)
│  └─ globals.css                1.026 linhas
├─ components/                   14 arquivos · 1.238 linhas
├─ lib/                          10 arquivos · 782 linhas
├─ hooks/                        useStorage · useCronometro
├─ data/                         planos e técnicas (conteúdo)
├─ public/                       sw.js · ícones · manifest · offline
├─ docs/                         fluxograma e relatório
└─ proxy.ts                      proteção das rotas
```

data	Conteúdo: planos e técnicas. Sem lógica de tela.
lib	Regras de negócio puras: datas, treinos, carga, alertas, sessão. Sem React, fáceis de testar.
hooks	Estado reutilizável: armazenamento local e cronômetro.
components	Interface compartilhada e o estado global do app.
app	Rotas e telas, que só montam as peças acima.

Pontos fortes

- Regras de cálculo separadas da interface
- Tipos e validadores num só lugar
- Dados de fora (armazenamento, backup) sempre validados

Pontos de concentração

- `CornerProvider.tsx`: 329 linhas e muitas responsabilidades
- `AppSheets.tsx`: 324 linhas e 84 estilos inline
- `globals.css`: 1.026 linhas para tudo

O QUE FOI CONFERIDO E O QUE NÃO FOI

Qualidade e verificação

VERIFICAÇÃO	RESULTADO	OBSERVAÇÃO
Typecheck (TypeScript estrito)	Passa	Sem erros em todo o projeto.
Lint (ESLint)	Passa	Sem avisos.
Build de produção	Passa	Todas as rotas geradas.
48 verificações de navegador	Passam	Scripts descartáveis, fora do repositório.
Testes automatizados no projeto	Não existe	Sem Vitest, Jest ou Playwright configurado.
Auditoria de dependências	3 achados	Next (crítica, via `next/og`, que o app não usa), sharp (alta) e source-map-js (alta).
Sons do cronômetro	Não ouvido	Testado sem tela: o fluxo está certo, mas ninguém ouviu o sino num celular.
Segunda fase no ar	Não publicada	Ainda não enviada ao GitHub nem à Vercel.

O que as 48 verificações cobriram

- Login, sair da conta e proteção das rotas
- Perfil de atleta e plano recomendado
- Check-in, ajuste do dia e reagendamento
- Cronômetro: preparo, rounds, descanso, pausa e fim
- Registro com duração real e dados de sparring
- Alertas, saúde, peso, lesões e carga
- Biblioteca, backup válido e inválido

Modo offline e privacidade

- Servidor desligado: Hoje e cronômetro abrem do aparelho
- Tela nunca visitada mostra a página offline
- Depois de sair da conta, o cache é apagado
- Lembrete diário disparou com o app aberto

As verificações de navegador provam o comportamento de hoje, mas não protegem o futuro: sem ficarem salvas no projeto, qualquer mudança nova pode quebrar algo sem ninguém perceber. Esse é o principal motivo da primeira sugestão da próxima página.

MELHORIAS ESPERADAS · PARTE 1

Sugestões para o processo

Combinados para o trabalho daqui para frente, com o ganho esperado de cada um.

SUGESTÃO	MELHORIA ESPERADA	ESFORÇO	PRIORIDADE
TDD com Vitest nas regras puras (`lib/` e `data/`)	Rede de segurança: erro numa regra de carga, prontidão, datas ou sequência é pego na hora, antes de chegar ao usuário. O roteiro das 48 verificações vira teste permanente.	Médio cerca de 1 dia	Alta
Atualizar o Next	Fecha a vulnerabilidade crítica. Fazer em um commit separado, com build e teste logo depois.	Baixo a médio	Alta
Auditoria antes de entregar (o equivalente ao `/doctor`)	Typecheck, lint, build, auditoria de dependências e conferência de variáveis de ambiente antes de cada envio. Evita repetir o problema do login na Vercel.	Baixo	Alta
Commits menores e frequentes	Hoje há 44 arquivos pendentes (16 alterados e 28 novos). Commits por assunto facilitam revisar, desfazer e entender o histórico.	Baixo	Média
CI no GitHub (Actions)	Roda typecheck, lint, testes e build a cada envio e bloqueia o que quebra. A Vercel só publica código que passou.	Baixo a médio	Média
Subagentes em tarefas grandes	Um escreve o código, outro os testes, outro audita, em paralelo. Menos tempo de espera e uma segunda leitura independente.	Baixo	Média
Teste em celular real a cada entrega	Pega o que o navegador sem tela não vê: som do sino, instalação do app, vibração (que não existe no iPhone) e a tela acesa.	Baixo	Média

Antes

Entregas verificadas à mão, scripts de teste descartados, problemas de produção descobertos pelo usuário.

Depois

Testes salvos, CI no GitHub, auditoria padronizada e commits pequenos.

Ganho

Menos retrabalho, mudanças mais seguras e produção estável.

MELHORIAS ESPERADAS · PARTE 2

Sugestões para a estrutura do código

Mudanças que não alteram o que o usuário vê, mas deixam o código mais fácil de manter e de evoluir.

SUGESTÃO	PROBLEMA DE HOJE	MELHORIA ESPERADA	PRIORIDADE
Dividir `CornerProvider` em partes menores (histórico, saúde, registro e painéis)	329 linhas: 9 blocos de dados guardados, painéis, backup e lembrete no mesmo lugar. Qualquer mudança no estado re-renderiza quem usa o contexto.	Leitura simples, menos re-renderização e testes por parte.	Média
Dividir `AppSheets`: um arquivo por painel, com CSS Module	324 linhas e 84 estilos inline misturados à lógica.	Estilo reaproveitável, painéis independentes e revisão mais fácil.	Média
Quebrar o `globals.css` em módulos por assunto	1.026 linhas globais: risco de uma regra afetar outra tela.	Estilos próximos do componente e remoção segura do que não é usado.	Baixa
Camada de dados única (repositório) e versão do esquema	As telas conhecem o armazenamento local e suas chaves.	Trocar o armazenamento local por uma API sem mexer nas telas, e migrar dados antigos com segurança.	Alta
Contrato de dados com validação central	Os validadores estão em `lib/tipos.ts` e no provider.	Uma só fonte para o que é um treino válido, usada no armazenamento, no backup e na futura API.	Média
Acessibilidade e textos revisados	Boa base (rótulos, foco, tabelas alternativas), mas sem revisão sistemática.	Checagem de contraste e navegação por teclado; textos num único lugar, abrindo caminho para outro idioma.	Baixa

Por que a camada de dados é prioridade

A conta com banco de dados é o próximo grande passo. Se as telas continuarem falando direto com o armazenamento do navegador, a troca vai mexer em quase todas. Com uma camada no meio, a troca fica em um ponto só.

Ordem sugerida

- Primeiro os testes (rede de segurança)
- Depois a camada de dados
- Por fim dividir provider e painéis, com os testes protegendo

O QUE VEM A SEGUIR

Riscos e próximos passos

Dados só no aparelho

Trocar de celular ou limpar o navegador apaga o histórico. Hoje a proteção é o backup em arquivo. A solução definitiva é a conta com banco de dados.

Senha de teste exposta

A senha do usuário de teste foi escrita na conversa. Antes de abrir o app a outras pessoas, troque-a e gere um novo hash e uma nova chave de sessão, também na Vercel.

Vulnerabilidades

Três achados nas dependências, um crítico (Next). O app não usa o recurso afetado, mas a atualização deve ser feita.

Segunda fase sem teste real

Som, instalação do app e vibração ainda não foram vistos num celular. A vibração não existe no iPhone.

Roteiro sugerido

Agora

- Auditoria e envio da segunda fase
- Instalar o Vitest e testar as regras de cálculo
- Atualizar o Next
- Trocar a senha de teste

Em seguida

- CI no GitHub
- Camada de dados única
- Testar no celular: som, instalação e tela acesa
- Dividir provider e painéis

Depois

- Conta com banco de dados e sincronização
- Notificações push com o app fechado
- Cadastro de novos usuários

Decisão pendente

Para avançar na conta com banco de dados, falta escolher o provedor (por exemplo, Supabase ou Neon) e criar as credenciais. Com isso definido, as notificações push e o cadastro de usuários passam a ser possíveis.

Documentos desta sessão

- `docs/Corner-Fluxograma.pdf`: o fluxo do app, em 8 páginas
- `docs/Corner-Relatorio.pdf`: este relatório
- `docs/fluxograma.html` e `docs/relatorio.html`: fontes editáveis